ТИИСПИС · ВАРИАНТ 26 · «ЯЗЫКИ ПРОГРАММИРОВАНИЯ»

Как защищать ЛР2 и ЛР3 на одной системе

В системе одна база знаний, в ней сразу обе лабораторные. Здесь: чем они отличаются, почему правил так много, как устроен каждый вид правила и что происходит внутри, когда в чат вводят вопрос.

ЧИСЛА, КОТОРЫЕ НАДО ЗНАТЬ

Правил распознавания в системе	**100 = 58 + 2 + 40**
Из них сделано в ЛР2	**60 без изменений, 7 заменены в ЛР3**
Правил ответа своих	**6: одно из ЛР2, пять из ЛР3**
Фраз своих	**12: две из ЛР2, десять из ЛР3**
Сборка базы	**`[1826/1826]`**

СОДЕРЖАНИЕ

1. Разница между ЛР2 и ЛР3
2. Что в системе от какой лабы
3. Что изменилось в части ЛР2
4. Почему правил так много
5. Все виды правил одной таблицей
6. Правило распознавания ЛР2
7. Правило распознавания ЛР3
8. Штатные правила ответа
9. Своё правило ответа
10. Правило «сведений нет»
11. Фразы: штатные и свои
12. Общая цепочка: кто за кем работает
13. Разбор 1. «Что такое компилятор?»
14. Разбор 2. «На что декомпозируется...»
15. Разбор 3. «Кто создал Python?»
16. Разбор 4. «Какая реализация у Python?»
17. Разбор 5. «Какая реализация у Haskell?»
18. Разбор 6. Вопрос без правила
19. Что показывать на ЛР2
20. Что показывать на ЛР3
21. Вопросы на стыке двух лаб

Пути к файлам даны от папки `nika/kb/`. «Своя папка» — `extra/section_subject_domain_of_programming_languages/`.

ЧАСТЬ I · ОБЩАЯ КАРТИНА

1. Разница между ЛР2 и ЛР3

ЛР2 — как система понимает вопрос. ЛР3 — как она строит ответ.

Сравнение

	ЛР2	ЛР3
Тема	классификация сообщения	логические правила ответа и фразы
На какой вопрос отвечает правило	«о чём это сообщение?»	«чем на него отвечать?»
Что в условии правила	текст сообщения совпал с образцом	у сообщения есть класс и сущность, и в базе есть данные
Что правило делает	вешает на сообщение класс и сущность	создаёт ответное сообщение и указывает класс фраз
В какое множество входит	`concept_classify_message_rule_priority_1`	`concept_answer_on_standard_message_rule_priority_2` или `_3`

	ЛР2	ЛР3
Какой агент исполняет	`AlternativeMessageTopicClassificationAgent`	`DirectInferenceAgent` внутри `StandardMessageReplyAgent`
Главное, что показать	вопрос распознан: у сообщения появились класс и `rrel_entity`	один и тот же вопрос даёт разные ответы в зависимости от базы

> ОДНОЙ ФРАЗОЙ
>
> Ни правило ЛР2, ни правило ЛР3 не пишет текст ответа. Первое ставит на сообщение ярлыки, второе выбирает набор фраз, а сам текст собирает третий механизм — агент генерации фраз.

2. Что в системе от какой лабы

База одна, но у каждого файла есть «хозяин».

Инвентарь

ЧТО	ЛР2	ЛР3
Вопросы	«Что такое …?» / «Кто такой …?» «На что декомпозируется …?» «Кто создал …?»	год создания, реализация, стандарт, организация; вопрос об авторе расширен
Правила распознавания	58 `lr_classify_what_is_*` 2 `lr_classify_subdividing_of_*` 7 про автора — заменены в ЛР3	40 `lr_classify_<тип>_of_<язык>`: 5 типов × 8 языков, включая 8 про автора
Классы сообщений	`concept_message_about_author`	`…about_creation_year`, `…about_implementation`, `…about_standard`, `…about_maintainer` и общий `concept_message_about_language_property`
Правила ответа	`lr_answer_about_author`	`lr_answer_about_creation_year`, `…implementation`, `…standard`, `…maintainer` и `lr_answer_about_unknown_language_property`
Фразы	2 фразы об авторе	8 фраз с шаблоном и 2 фразы «сведений нет»

Правило ответа и фразы об авторе сделаны в ЛР2 заранее: без них вопрос «Кто создал …?» распознавался бы, но ответа не было бы. В отчёте ЛР2 это так и написано.

3. Что изменилось в части ЛР2

Из сделанного в ЛР2 тронуты только семь правил про автора. Отсюда четыре следствия.

Четыре отличия от отчёта ЛР2

1. **Формулировок про автора стало четыре вместо двух.** Было «Кто создал X?» и «Кто автор X?». Добавились «Кто разработал X?» и «Кто создал X» без знака вопроса.
2. **В следствии этих правил появился второй класс** — общий `concept_message_about_language_property`. Он нужен правилу «сведений нет» из ЛР3.
3. **«Кто автор Пролог?» стало «Кто автор Пролога?»** — название поставлено в родительный падеж.
4. **Добавлено восьмое правило про автора — для языка ассемблера x86.** В отчёте ЛР2 вопрос «Кто создал язык ассемблера x86?» приведён как пример вопроса без правила. Теперь правило есть, и ответ другой — одна из двух фраз «сведений нет», например «В базе знаний нет таких сведений про язык ассемблера x86».

> ЧТО ИЗ ЭТОГО СЛЕДУЕТ НА ЗАЩИТЕ
>
> Как пример нераспознанного вопроса задавать не «Кто создал язык ассемблера x86?», а «Какой компилятор у Python?» — такого типа вопроса нет вообще.

Что не менялось

58 правил «Что такое ...?», 2 правила про декомпозицию, класс сообщения об авторе, правило ответа об авторе и две фразы об авторе совпадают с состоянием ЛР2 байт в байт. Файлы первой лабораторной тоже не тронуты.

Про числа

В отчёте ЛР2 и на его рисунках — 67 правил и 134 формулировки. В системе сейчас 100 правил.

> СКАЗАТЬ ВСЛУХ
>
> 67 правил было на момент второй лабораторной. В третьей семь правил про автора заменены расширенными и добавлены 33 новых — всего стало 100.

4. Почему правил так много

Да: на каждое понятие и на каждое сочетание «вопрос × язык» — своё правило. Это не лишняя работа, а свойство механизма.

Откуда 100

ГРУППА	СКОЛЬКО	КАК ПОЛУЧИЛОСЬ
«Что такое ...?» / «Кто такой ...?»	58	по одному на каждую сущность: 23 класса и 35 экземпляров
«На что декомпозируется ...?»	2	по одному на каждое понятие, у которого в базе записано разбиение
5 типов вопроса о языке	40	5 типов × 8 языков

Вместе со штатными правилами NIKA агент перебирает 113 правил на каждое сообщение — это видно в журнале: `There is 113 formulas in 1 set`.

Почему нельзя одним правилом

Правило распознавания устроено так: «если текст сообщения **равен** вот этой строке, то сущность сообщения — **вот этот узел**». В нём жёстко записаны и строка, и узел.

- **Строка сравнивается целиком и посимвольно.** Механизм не умеет вынуть из текста слово «компилятор» и подставить его куда-то. Он умеет только ответить «совпало» или «не совпало».
- **Сущность в следствии — конкретный узел базы.** Чтобы по тексту «Что такое интерпретатор?» получить узел `concept_interpreter`, эту пару надо где-то записать. Записана она в правиле.

Поэтому «Что такое компилятор?» и «Что такое интерпретатор?» — два разных правила: разные строки и разные узлы. В самой NIKA сделано так же — в базе есть штатное правило `lr_classify_what_is_nika_message` для вопросов «Что такое Ника?» и «Кто такая Ника?», и больше ни для чего. Свои правила получены из него.

Кто должен был это обобщать

Обобщать формулировки умеет Wit.ai: его обучают на примерах, и он сам находит в новой фразе намерение и сущность. Тогда хватило бы одного класса сообщений на тип вопроса. Но штатный токен Wit.ai сервис отвергает, поэтому распознавание сделано правилами, а у правил обобщения нет.

> СКАЗАТЬ ВСЛУХ
>
> Правило классификации сопоставляет конкретный текст с конкретной сущностью, поэтому на каждую сущность и каждый тип вопроса нужно своё правило. Все они построены по одному образцу и отличаются тремя вещами: именем, текстами вопроса и тем, что стоит в следствии. Так же устроены штатные правила NIKA.

Если спросят, писались ли они руками: нет, они получены из одного рабочего правила подстановкой имени, текстов и сущности. Скрывать это незачем — сто одинаковых по форме файлов руками никто не пишет.

5. Все виды правил одной таблицей

В ответе на один вопрос участвуют два правила: распознавания и ответа. Видов тех и других немного.

Восемь видов

ВИД ПРАВИЛА	ЧЬЁ	СКОЛЬКО	МНОЖЕСТВО	ГДЕ ЛЕЖИТ
Распознавание «Что такое …?»	ЛР2	58	классификация, приоритет 1	своя папка, `logic_rules/classification/`
Распознавание «На что декомпозируется …?»	ЛР2	2	то же	там же
Распознавание «тип × язык»	ЛР3	40	то же	там же
Ответ о сущности `lr_message_about_entity`	штатное	1	ответ, приоритет 2	`section_subject_domain_of_dialogues/…/logic_rules/`
Ответ о разбиении `lr_message_about_subdividing`	штатное	1	ответ, приоритет 1	там же
Ответ о свойстве языка `lr_answer_about_*`	ЛР2: автор ЛР3: ещё четыре	5	ответ, приоритет 2	своя папка, `logic_rules/answer/`
«Сведений нет» `lr_answer_about_unknown_language_property`	ЛР3	1	ответ, приоритет 3	там же
Непонимание `lr_not_classified_by_intent_message`	штатное	1	ответ, приоритет 1	`section_subject_domain_of_dialogues/…/logic_rules/`

Все правила — файлы `.gwf`. Каждое правило — импликация: контур-условие, контур-следствие и связь между ними.

6. Правило распознавания ЛР2

Файл `logic_rules/classification/lr_classify_what_is_compiler_message.gwf`. Разбор по настоящей структуре файла.

Три контура

```
if_what_is_compiler_1     условие: текст = «Что такое компилятор?»
if_what_is_compiler_2     условие: текст = «Что значит компилятор?»
(без имени)               следствие
```

Что внутри контура-условия

```
concept_message            ⟶  _message                1. это сообщение
(безымянная переменная)    ⟹  _message                2. у него есть текстовое представление
nrel_sc_text_translation   ⟶  (связь из строки 2)        …связанное отношением «трансляция sc-текста»
(безымянная переменная)    ⟶  «Что такое компилятор?» 3. в представлении лежит ссылка с таким текстом
```

Читается: «существует сообщение, у которого есть текстовое представление, а в нём — ссылка с содержимым „Что такое компилятор?“».

- `concept_message`, `nrel_sc_text_translation` — **константы**: конкретные узлы базы.
- `_message` и безымянный узел — **переменные**: на их место система подставляет то, что найдёт.
- Ссылка с текстом тоже переменная, но с содержимым. Пометка `concept_template_with_links` на контуре говорит агенту: сравнивай содержимое ссылки.

Что внутри контура-следствия

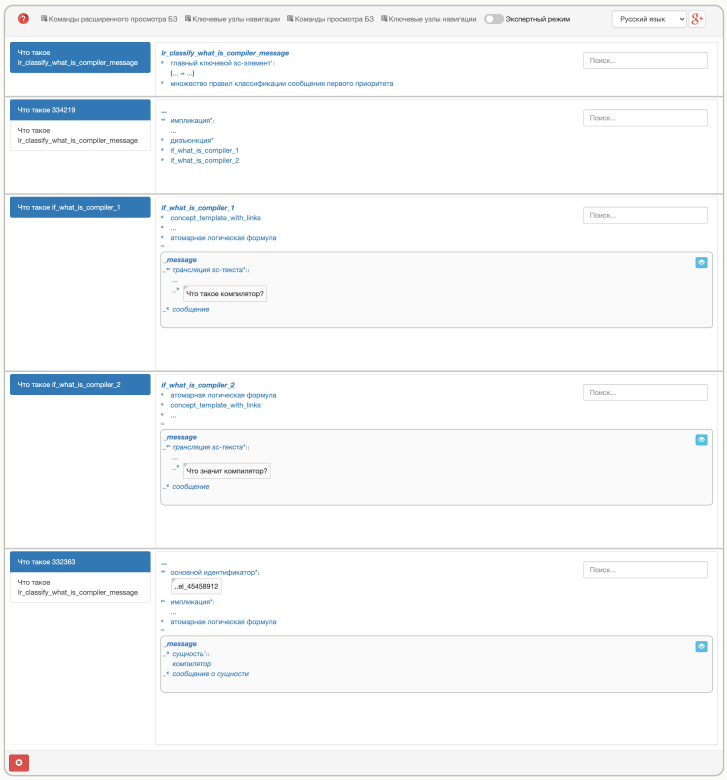
```
concept_message_about_entity  ⟶  _message            сообщение относится к классу «сообщение о сущности»
_message                      ⟶  concept_compiler    у сообщения есть элемент — компилятор
rrel_entity                   ⟶  (эта связь)         и роль этого элемента — «сущность'»
```

Переменная `_message` та же, что в условии. Раз в условии она уже связалась с нашим сообщением, следствие дорисовывает дуги именно к нему.

Обвязка: что делает набор узлов правилом

```
кортеж  ⟶  if_what_is_compiler_1 ; if_what_is_compiler_2   оба условия собраны в кортеж
nrel_disjunction  ⟶  кортеж                                кортеж — дизъюнкция: «или»
кортеж  ⟹  контур-следствие                                сама связь «если … то …»
nrel_implication  ⟶  (эта связь)                           она помечена как импликация
lr_classify_what_is_compiler_message  ⟶  (эта связь)       импликация принадлежит правилу
rrel_main_key_sc_element  ⟶  (предыдущая дуга)             …как его главный ключевой элемент
atomic_logical_formula  ⟶  каждый из трёх контуров         каждый контур — атомарная формула
concept_classify_message_rule_priority_1  ⟶  правило       правило входит в множество правил классификации
```

Целиком: «если текст совпал с первым **или** вторым образцом, то отнести сообщение к классу и привязать сущность; это правило классификации первого приоритета».



То же самое в sc-web: правило, дизъюнкция, два условия с текстами, следствие.

Правило про декомпозицию — то же самое с другим классом

Файл `lr_classify_subdividing_of_programming_lang_message.gwf`. Структура один в один, отличий три:

	«Что такое компилятор?»	«На что декомпозируется язык программирования?»
Тексты	Что такое компилятор? / Что значит компилятор?	На что декомпозируется язык программирования? / Каким бывает язык программирования?
Класс в следствии	`concept_message_about_entity`	`concept_message_about_subdividing`
Сущность	`concept_compiler`	`concept_programming_language`

Оба класса сообщений штатные. Поэтому для них правила ответа и фразы в NIKA уже есть.

7. Правило распознавания ЛР3

Файл `logic_rules/classification/lr_classify_implementation_of_python_lang_message.gwf`. Та же схема, два отличия.

Пять контуров вместо трёх

```
if_implementation_of_python_lang_1   «Какая реализация у Python?»
if_implementation_of_python_lang_2   «Какая реализация используется для Python?»
if_implementation_of_python_lang_3   «Чем реализован Python?»
if_implementation_of_python_lang_4   «Какая реализация у Python»        ← без знака вопроса
(без имени)                          следствие
```

Каждый контур-условие устроен точно так же, как в ЛР2. Отличие первое: условий четыре, все в одном кортеже-дизъюнкции.

В следствии два класса

```
concept_message_about_implementation     ⟶  _message          частный класс: вопрос о реализации
concept_message_about_language_property  ⟶  _message          общий класс: вопрос о свойстве языка
_message                                 ⟶  python_language   сущность — экземпляр из ЛР1
rrel_entity                              ⟶  (эта связь)
```

Отличие второе: сообщение получает сразу два класса. Частный нужен правилу ответа по существу, общий — правилу «сведений нет».

> **ПОЧЕМУ ОБЩИЙ КЛАСС СТАВИТСЯ ЯВНО**
>
> В базе записано, что «сообщение о реализации» включено в «сообщение о свойстве языка». Но принадлежность по включению сама не распространяется: если дуги от общего класса к сообщению нет, поиск её не найдёт. Поэтому правило создаёт обе дуги.

Сорок правил: что меняется от одного к другому

ТИП ВОПРОСА	ЧАСТНЫЙ КЛАСС	ПРИМЕР ФОРМУЛИРОВКИ
`author`	`concept_message_about_author`	Кто создал Python?
`creation_year`	`concept_message_about_creation_year`	В каком году создан Python?
`implementation`	`concept_message_about_implementation`	Какая реализация у Python?
`standard`	`concept_message_about_standard`	Какой стандарт у Python?
`maintainer`	`concept_message_about_maintainer`	Кто сопровождает Python?

Имя файла — `lr_classify_<тип>_of_<язык>_message.gwf`, язык — `python_lang`, `c_lang`, `cpp_lang`, `java_lang`, `javascript_lang`, `haskell_lang`, `prolog_lang`, `assembly_lang_x86`.

8. Штатные правила ответа

Они были в NIKA. На них держатся ответы на вопросы «Что такое ...?» и «На что декомпозируется ...?» из ЛР2.

`lr_message_about_entity` — ответ о сущности

```
ЕСЛИ  _message ∈ concept_message
  И   _message ∈ concept_message_about_entity
  И   _message → rrel_entity → _entity
ТО    _reply_message ∈ concept_message_about_entity
      _reply_message ∈ concept_atomic_message
      (_message → _reply_message) ∈ nrel_reply

приоритет: 2        класс фраз: concept_phrase_about_entity
```

Условий три. Проверки «есть ли у сущности описание» в нём нет — правилу достаточно, что сущность выделена.

`lr_message_about_subdividing` — ответ о разбиении

```
ЕСЛИ  _message ∈ concept_message
  И   _message ∈ concept_message_about_subdividing
  И   _message → rrel_entity → _entity
ТО    создать ответное сообщение и связать его с вопросом

приоритет: 1        класс фраз: concept_phrase_about_subdividing
```

`lr_not_classified_by_intent_message` — непонимание

```
ЕСЛИ  _message ∈ concept_message
  И   _message ∈ concept_not_classified_by_intent_message
ТО    создать ответное сообщение класса concept_misunderstanding_message

приоритет: 1        класс фраз: concept_misunderstanding_phrase
```

Срабатывает, когда сообщение не узнало ни одно правило и не помог Wit.ai. В классе фраз — «Сформулируйте, пожалуйста, по-другому», «Я не понимаю Вас», «Извините, я не знаю, как ответить», «К сожалению, я не могу ответить на Ваше сообщение».

9. Своё правило ответа

Файл `logic_rules/answer/lr_answer_about_implementation.gwf`. Так же устроены правила про автора, год, стандарт и организацию.

Условие и следствие

```
ЕСЛИ  _message ∈ concept_message                          1. это сообщение
  И   _message ∈ concept_message_about_implementation     2. это вопрос о реализации
  И   _message → rrel_entity → _entity                    3. в нём выделена сущность
  И   _entity → nrel_language_implementation → _value     4. у этой сущности в базе ЕСТЬ реализация
ТО    _reply_message ∈ concept_message_about_entity
      _reply_message ∈ concept_atomic_message
      (_message → _reply_message) ∈ nrel_reply

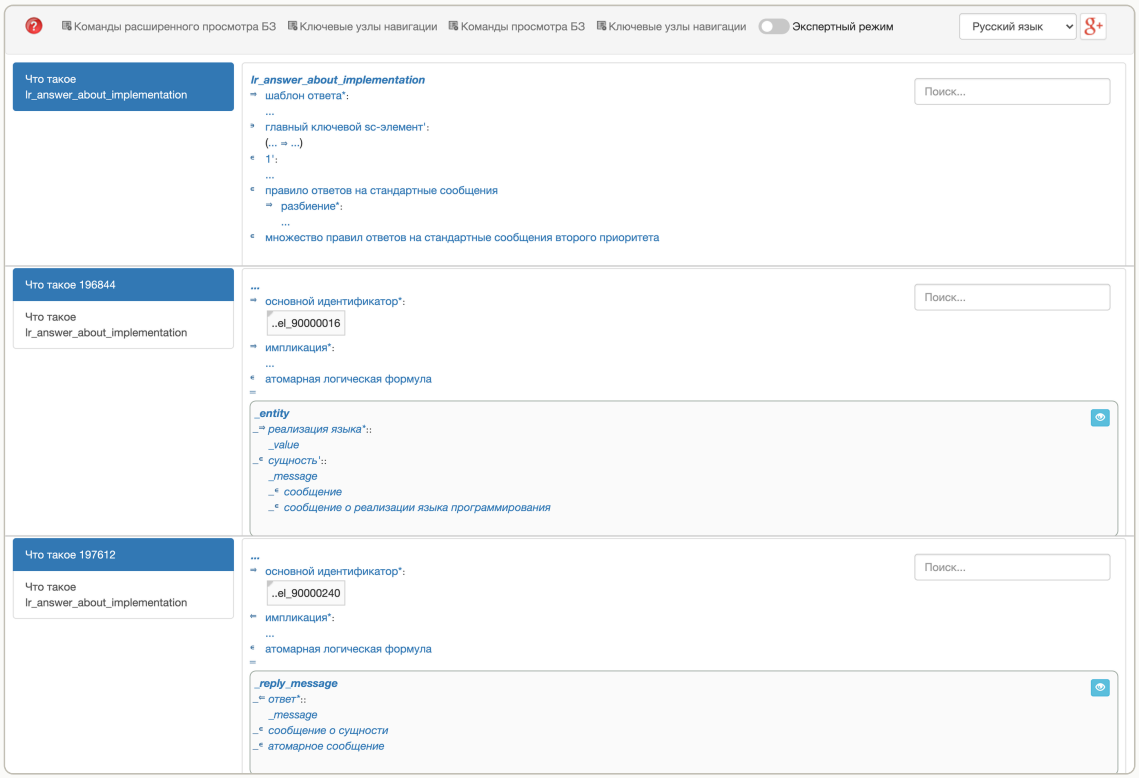
приоритет: 2        класс фраз: concept_phrase_about_implementation
```

Все четыре условия лежат в одном контуре. Элементы одного контура связаны «и», поэтому правило срабатывает, только когда верны все четыре.

Чем оно отличается от штатного

Ровно одним — **четвёртым условием**. У штатного `lr_message_about_entity` условий три, у своего четыре: добавлена проверка, что у сущности есть нужное отношение.

Переменная `_value` больше нигде не используется. Она нужна только как проверка существования: «найди хоть что-нибудь, с чем этот язык связан отношением `nrel_language_implementation`». Зачем — следующий раздел.



Правило в sc-web: само правило с приоритетом, контур-условие с четырьмя утверждениями, контур-следствие.

Пять правил: что меняется

ПРАВИЛО	КЛАСС СООБЩЕНИЯ В УСЛОВИИ 2	ОТНОШЕНИЕ В УСЛОВИИ 4	ЛАБА
`lr_answer_about_author`	`…about_author`	`nrel_language_author`	ЛР2
`lr_answer_about_creation_year`	`…about_creation_year`	`nrel_creation_year`	ЛР3
`lr_answer_about_implementation`	`…about_implementation`	`nrel_language_implementation`	ЛР3
`lr_answer_about_standard`	`…about_standard`	`nrel_language_standard`	ЛР3
`lr_answer_about_maintainer`	`…about_maintainer`	`nrel_maintaining_organization`	ЛР3

Класс фраз называется так же, как правило: `lr_answer_about_X` → `concept_phrase_about_X`.

10. Правило «сведений нет»

Файл `logic_rules/answer/lr_answer_about_unknown_language_property.gwf`. Одно на все пять типов вопроса.

Условие и следствие

```
ЕСЛИ  _message ∈ concept_message
  И   _message ∈ concept_message_about_language_property   любой вопрос о свойстве языка
ТО    создать ответное сообщение и связать его с вопросом

приоритет: 3        класс фраз: concept_phrase_about_unknown_language_property
```

Условий всего два, сущность не проверяется. Правилу неважно, о каком свойстве спросили.

Как оно взаимодействует с правилами второго приоритета

Агент вывода проверяет множества по порядку: сначала первый приоритет, потом второй, потом третий. И **останавливается на первом сработавшем правиле** — в журнале это строка `Target is achieved`.

ВОПРОС	ПРАВИЛО ВТОРОГО ПРИОРИТЕТА	ЧТО ДАЛЬШЕ
Какая реализация у Python?	сработало: у Python реализация есть	перебор остановлен, ответ по существу
Какая реализация у Haskell?	не сработало: четвёртое условие ложно	перебор дошёл до третьего приоритета, ответ «сведений нет»

> **ЗАЧЕМ ЧЕТВЁРТОЕ УСЛОВИЕ**
>
> Без него правило второго приоритета сработало бы и для Haskell, перебор остановился бы, фраза с шаблоном не собралась бы — и ответа не было бы вовсе. До правила третьего приоритета вывод уже не дошёл бы. Проверка наличия данных в самом правиле — главная находка третьей лабораторной.

11. Фразы: штатные и свои

Правило ответа называет класс фраз. Агент генерации берёт из класса фразу и заполняет пропуски.

Штатные фразы о сущности

Файл `section_subject_domain_of_messages/subject_domain_of_phrases/examples/concept_phrase_about_entity/concept_phrase_about_entity.gwf`. В классе две фразы, у каждой свой образец:

ФРАЗА	ЧТО ИЩЕТ ОБРАЗЕЦ	СОБИРАЕТСЯ У НАС?
`${_entity_definition_name} - ${_definition}.`	сущность → `nrel_definition` → текст	**нет: отношения `nrel_definition` у наших понятий нет**
`${_entity_note_name} - ${_note}.`	сущность → `nrel_note` → текст	**да: примечание есть у каждого понятия**

Порядок, в котором система берёт фразы класса, не фиксирован. Отсюда три вещи, которые видно на экране и в журнале:

- если первой взята фраза с определением, в журнале появляется строка `[Error]: Answer isn't found.` — фраза не собралась, после неё берётся вторая. Если первой взята фраза с примечанием, этой строки нет. Ответ в обоих случаях одинаковый;
- ответ начинается с названия: «компилятор - Компилятор — программа...» — фраза ставит название, дефис и примечание, а примечание само начинается с названия;
- в конце две точки: фраза добавляет точку к примечанию, которое уже заканчивается точкой.

Штатная фраза о разбиении

Файл `…/examples/concept_phrase_about_subdividing/concept_phrase_about_subdividing.gwf`. В классе две фразы. «... декомпозируется на ...» у нас не собирается, и в журнале после неё стоит `Answer isn't found`. Собирается вторая — `${_subdivision_entity_name} разбивается на $...{_subdividing}.` Запись `$...{…}` означает «перечислить все элементы множества».

У языка программирования в базе четыре разбиения. Фраза выводит элементы одного из них — поэтому в ответе четыре вида языков, а не все десять.

Свои фразы

Файлы `phrases/examples/concept_phrase_about_*.scs` в своей папке. В каждом — объявление двух переменных, один образец и две фразы. Образец у всех одной формы:

```
сообщение ──rrel_entity──▶ язык
                           ├──nrel_main_idtf──▶ его русское название          → первый пропуск
                           └──отношение──────▶ значение
                                               └──nrel_main_idtf──▶ его название → второй пропуск
```

КЛАСС ФРАЗ	ОТНОШЕНИЕ В ОБРАЗЦЕ	ФРАЗЫ
`…about_author`	`nrel_language_author`	Автор языка программирования … — …. Язык программирования … создал ….
`…about_creation_year`	`nrel_creation_year`	Год создания языка программирования … — …. …: год создания — ….
`…about_implementation`	`nrel_language_implementation`	Реализация языка программирования … — …. Для языка программирования … используется реализация «…».
`…about_standard`	`nrel_language_standard`	Стандарт языка программирования … — …. Язык программирования … описан стандартом ….
`…about_maintainer`	`nrel_maintaining_organization`	Язык программирования … сопровождает …. Сопровождением языка программирования … занимается ….
`…about_unknown_language_property`	общий образец: только название языка	В базе знаний нет таких сведений про …. Этого про … я пока не знаю.

Система берёт любую из двух фраз класса, поэтому на один и тот же вопрос формулировка бывает разной.

ЧАСТЬ III · ЧТО ПРОИСХОДИТ, КОГДА ВВОДИМ ТЕКСТ

12. Общая цепочка: кто за кем работает

Порядок шагов записан в базе, в файле `section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/examples/message_processing_program.gwf`.

Шаги обработки любого сообщения

1. **Окно чата создаёт сообщение в базе**
   Узел-сообщение, его текстовое представление и ссылка с текстом. Запускается `MessageReplyAgent`, он запускает программу обработки.
2. **Распознавание по правилам — `AlternativeMessageTopicClassificationAgent`**
   Перебирает 113 правил классификации. Здесь работают правила ЛР2 и ЛР3. Итог — `Message is classified` либо `Message is not classified`.
3. **Wit.ai — `MessageTopicClassificationAgent`, только если правила не помогли**
   Сервис отвечает `no-auth`, сообщение помечается как нераспознанное.
4. **Штатные агенты поиска слова по букве и погоды**
   Видят, что вопрос не их, и сразу завершаются. Агент погоды работает в контейнере `py-sc-server` — поэтому без него цепочка встаёт.
5. **Ответ — `StandardMessageReplyAgent`**
   Запускает `DirectInferenceAgent`: тот перебирает правила ответа по приоритетам и останавливается на первом сработавшем. Потом берётся класс фраз, и `PhraseGenerationAgent` собирает текст.
6. **Окно чата показывает ответ**

Дальше шесть разборов по этой цепочке. Серые блоки — строки из настоящего журнала решателя, в сокращении: многоточие заменяет штатные правила, которые проверены и не подошли.

> ПРО ПОРЯДОК
>
> Приоритеты всегда идут по порядку: первый, второй, третий. А внутри одного приоритета порядок перебора правил система выбирает сама, и после пересборки базы он может быть другим. Поэтому в вашем журнале правила одного приоритета могут стоять не так, как здесь, — на результат это не влияет.

Журнал открывается командой `docker compose logs -f problem-solver` из папки `nika`.

13. Разбор 1. «Что такое компилятор?»

ЛР2. Своё правило распознавания, штатное правило ответа, штатная фраза.

По шагам

1 **Распознавание**

Файл: `logic_rules/classification/lr_classify_what_is_compiler_message.gwf`.
Совпало первое условие, `if_what_is_compiler_1`: текст сообщения равен «Что такое компилятор?».
Что добавлено в базу: сообщение ∈ `concept_message_about_entity`; сообщение → `rrel_entity` → `concept_compiler`.

```
There is 113 formulas in 1 set
Compute atomic logical formula if_what_is_compiler_1 true
Message is classified.
```

2 **Выбор правила ответа**

Первый приоритет — десять штатных правил, ни одно не подошло. Второй приоритет: три своих правила о свойствах языка не подошли по классу сообщения, штатное `lr_message_about_entity` сработало.
Файл: `section_subject_domain_of_dialogues/…/logic_rules/lr_message_about_entity/lr_message_about_entity.gwf`.
Подстановка: `_entity` → `concept_compiler`.

```
There is 10 formulas in 1 set
Trying to generate by formula: lr_message_about_subdividing
…
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
Trying to generate by formula: lr_message_about_entity
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_message_about_entity is found
```

3 **Фраза**

Файл: `section_subject_domain_of_messages/…/concept_phrase_about_entity.gwf`.
Фраза с определением не собирается: у понятия нет `nrel_definition`. Фраза с примечанием собирается.
Откуда слова: файл `concepts/concept_compiler.scs` — русское название «компилятор» и примечание `nrel_note`.

```
MessageHandler: the phrase with the content "${_entity_definition_name} - ${_definition}." is found
[Error]: Answer isn't found.
MessageHandler: the phrase with the content "${_entity_note_name} - ${_note}." is found
Generated text: "компилятор - Компилятор — программа, переводящая исходный текст…"
```

> **ДЛЯ ЗАЩИТЫ ЛР2**
> Своё здесь — только шаг 1. Шаги 2 и 3 выполняют штатные правило и фраза. Это и есть содержание второй лабораторной: научить систему распознавать вопрос.

14. Разбор 2. «На что декомпозируется язык программирования?»

ЛР2. Отличается от первого разбора классом сообщения и приоритетом правила ответа.

По шагам

1. **Распознавание**

   Файл: `logic_rules/classification/lr_classify_subdividing_of_programming_lang_message.gwf`.

   Что добавлено: сообщение ∈ `concept_message_about_subdividing`; сущность — `concept_programming_language`.

   ```
   Compute atomic logical formula if_subdividing_of_programming_lang_1 true
   Message is classified.
   ```

2. **Выбор правила ответа**

   Штатное `lr_message_about_subdividing` стоит в **первом** приоритете и срабатывает сразу.

   ```
   There is 10 formulas in 1 set
   Trying to generate by formula: lr_message_about_subdividing
   Compute atomic logical formula  true
   Target is achieved
   ```

3. **Фраза**

   Фраза «... декомпозируется на ...» не собирается. Фраза «... разбивается на ...» собирается.

   Откуда слова: файл `concepts/concept_programming_language.scs` — одно из четырёх разбиений `nrel_subdividing` и названия его элементов.

   ```
   MessageHandler: the logic rule lr_message_about_subdividing is found
   MessageHandler: the phrase with the content "${_decomposition_entity_name} декомпозируется на $...{_decomposition}." is found
   [Error]: Answer isn't found.
   MessageHandler: the phrase with the content "${_subdivision_entity_name} разбивается на $...{_subdividing}." is found
   Generated text: "язык программирования разбивается на логический язык программирования, …"
   ```

15. Разбор 3. «Кто создал Python?»

Вопрос из ЛР2, но в общей системе его правило распознавания уже в виде ЛР3. Правило ответа и фразы — из ЛР2.

По шагам

1. **Распознавание**

   Файл: `logic_rules/classification/lr_classify_author_of_python_lang_message.gwf` — заменён в ЛР3, условий четыре.

   Что добавлено: сообщение ∈ `concept_message_about_author`; сообщение ∈ `concept_message_about_language_property`; сущность — `python_language`.

   ```
   Compute atomic logical formula if_author_of_python_lang_1 true
   Message is classified.
   ```

2 **Выбор правила ответа**

Файл: `logic_rules/answer/lr_answer_about_author.gwf`, второй приоритет.
Штатное `lr_message_about_entity` тоже проверяется, но не срабатывает: класса «сообщение о сущности» у этого сообщения нет.
Проверка четырёх условий: это сообщение — да; вопрос об авторе — да; сущность есть, `_entity` → `python_language`; автор записан, `_value` → `guido_van_rossum`.

```
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
Trying to generate by formula: lr_message_about_entity
…
Trying to generate by formula: lr_answer_about_creation_year
Trying to generate by formula: lr_answer_about_author
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_answer_about_author is found
```

3 **Фраза**

Файл: `phrases/examples/concept_phrase_about_author.scs`.
Откуда слова: `concepts/instances/languages/python_language.scs` — название «Python» и связь `nrel_language_author`; `concepts/instances/developers/guido_van_rossum.scs` — название «Гвидо ван Россум».

```
MessageHandler: the phrase with the content "Автор языка программирования ${_language_name} — ${_author_name}." is found
Generated text: "Автор языка программирования Python — Гвидо ван Россум."
```

> ДЛЯ ЗАЩИТЫ ЛР2
>
> Это единственный из трёх типов вопроса ЛР2, где своё всё: класс сообщения, правило распознавания, правило ответа и фразы. Если показываете файл правила распознавания — помните, что в нём уже четыре формулировки и два класса в следствии.

16. Разбор 4. «Какая реализация у Python?»

ЛР3. Всё своё, данные в базе есть.

По шагам

1 **Распознавание**

Файл: `logic_rules/classification/lr_classify_implementation_of_python_lang_message.gwf`.
Что добавлено: сообщение ∈ `concept_message_about_implementation`; сообщение ∈ `concept_message_about_language_property`; сущность — `python_language`.

```
Compute atomic logical formula if_implementation_of_python_lang_1 true
Message is classified.
```

2 **Выбор правила ответа**

Файл: `logic_rules/answer/lr_answer_about_implementation.gwf`.
Правила про организацию и стандарт не подошли по классу сообщения. У правила про реализацию сошлись все четыре условия: `_entity` → `python_language`, `_value` → `cpython_interpreter`.

```
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_answer_about_implementation is found
```

3 **Фраза**

Файл: `phrases/examples/concept_phrase_about_implementation.scs`.

Подстановка: `${_language_name_impl}` → «Python» из `python_language.scs`; `${_impl_name}` → «CPython» из `concepts/instances/implementations/cpython_interpreter.scs`.

```
MessageHandler: the phrase with the content "Реализация языка программирования ${_language_name_impl} — ${_impl_name}." is found
Generated text: "Реализация языка программирования Python — CPython."
```

17. Разбор 5. «Какая реализация у Haskell?»

ЛР3. Тот же вопрос, но данных в базе нет. Главная демонстрация третьей лабораторной.

По шагам

1 **Распознавание**

Файл: `lr_classify_implementation_of_haskell_lang_message.gwf`.

Сообщение получает те же два класса, сущность — `haskell_language`. Распознавание проходит так же, как для Python.

```
Compute atomic logical formula if_implementation_of_haskell_lang_1 true
Message is classified.
```

2 **Второй приоритет — ничего не сработало**

Правило про реализацию проверено, но четвёртое условие ложно: в файле `concepts/instances/languages/haskell_language.scs` отношения `nrel_language_implementation` нет. Остальные правила не подходят по классу сообщения.

```
There is 7 formulas in 2 set
Trying to generate by formula: lr_answer_about_maintainer
Trying to generate by formula: lr_answer_about_standard
Trying to generate by formula: lr_answer_about_implementation
Trying to generate by formula: lr_message_about_entity
…
Trying to generate by formula: lr_answer_about_creation_year
Trying to generate by formula: lr_answer_about_author
```

Ни после одной строки нет `Compute atomic logical formula true` — значит, не сработало ни одно.

3 **Третий приоритет — сработало правило «сведений нет»**

Файл: `logic_rules/answer/lr_answer_about_unknown_language_property.gwf`.

Его условие — сообщение принадлежит общему классу `concept_message_about_language_property`. Эту дугу создало правило распознавания на шаге 1.

```
There is 5 formulas in 3 set
…
Trying to generate by formula: lr_answer_about_unknown_language_property
Compute atomic logical formula  true
Target is achieved
MessageHandler: the logic rule lr_answer_about_unknown_language_property is found
```

4 **Фраза с общим шаблоном**

Файл: `phrases/examples/concept_phrase_about_unknown_language_property.scs`. Образец достаёт из базы одно — название языка из сообщения. Он общий для всех пяти типов вопроса: отношения, о котором спросили, в нём нет.

```
MessageHandler: the phrase with the content "Этого про ${_language_name_unknown} я пока не знаю." is found
Generated text: "Этого про Haskell я пока не знаю."
```

СРАВНИТЕ С РАЗБОРОМ 4

Шаг 1 одинаков. Расходятся разборы на шаге 2, и причина одна — строка в файле экземпляра: у Python она есть, у Haskell нет. Ни правила, ни фразы при этом не менялись.

18. Разбор 6. Вопрос без правила

Пример: «Какой компилятор у Python?» — такого типа вопроса нет. То же будет с вопросом, набранным с маленькой буквы.

По шагам

1. **Правила не помогли**

   Все 113 правил проверены, текст не совпал ни с одним образцом.

   ```
   There is 113 formulas in 1 set
   Message is not classified.
   ```

2. **Попытка через Wit.ai**

   Запускается штатный агент Wit.ai. Сервис отвергает токен, класс и сущности не найдены.

   ```
   MessageTopicClassificationAgent started
   {"code":"no-auth","error":"Invalid user: FB log-in user is no longer supported"}
   [Warning]: MessageTopicClassifier: Message intent class is not found.
   [Warning]: MessageTopicClassifier: Message entities are not found.
   ```

3. **Штатное правило непонимания**

   Сообщение помечено как нераспознанное, и в первом приоритете срабатывает `lr_not_classified_by_intent_message`.

   ```
   There is 10 formulas in 1 set
   Trying to generate by formula: lr_message_about_subdividing
   …
   Trying to generate by formula: lr_not_classified_by_intent_message
   Compute atomic logical formula  true
   Target is achieved
   MessageHandler: the logic rule lr_not_classified_by_intent_message is found
   ```

4. **Одна из четырёх штатных фраз непонимания**

   ```
   Generated text: "Сформулируйте, пожалуйста, по-другому."
   ```

ЗАЧЕМ ЭТО ЗНАТЬ

Строка `no-auth` в журнале — не поломка. Она появляется каждый раз, когда вопрос не узнало ни одно правило. Это же доказывает порядок: сначала правила, и только потом Wit.ai.

Чем «сведений нет» отличается от «непонимания»

	НЕПОНИМАНИЕ	«СВЕДЕНИЙ НЕТ»
Пример	Какой компилятор у Python?	Какая реализация у Haskell?
Распознан ли вопрос	**нет**	**да: известны тип вопроса и язык**
Какое правило отвечает	штатное `lr_not_classified_by_intent_message`, приоритет 1	своё `lr_answer_about_unknown_language_property`, приоритет 3
Что отвечает	«Сформулируйте, пожалуйста, по-другому» и подобное	«В базе знаний нет таких сведений про Haskell» или «Этого про Haskell я пока не знаю»

ЧАСТЬ IV · ЗАЩИТА

19. Что показывать на ЛР2

Тема — классификация. Показывать то, что не менялось и совпадает с отчётом.

Диалог

```
Что такое компилятор?
Кто создал Python?
На что декомпозируется язык программирования?
Что такое виртуальная машина?
```

Это сценарий из отчёта ЛР2. Отправлять кнопкой с самолётиком, по одному, дожидаясь ответа.

Что открыть в sc-web

Адрес — `http://localhost:8000/?sys_id=` и идентификатор.

ИДЕНТИФИКАТОР	ЧТО СКАЗАТЬ
`lr_classify_what_is_compiler_message`	«Правило классификации — импликация. Входит во множество правил первого приоритета».
`if_what_is_compiler_1`, `if_what_is_compiler_2`	«Посылка — дизъюнкция двух условий. В каждом — сообщение, у которого текст равен образцу».
`concept_message_about_author`	«Собственный класс сообщений. Два других класса — штатные».
`concept_classify_message_rule_priority_1`	«Множество, из которого агент берёт правила».
`concept_message_about_author` после вопроса про Python → щёлкнуть по сообщению «...»	«Результат классификации: у сообщения сущность' — Python».

> **КАКОЕ ПРАВИЛО ПОКАЗЫВАТЬ**
> Правило «Что такое компилятор?» — оно не менялось и совпадает с рисунком в отчёте. Правило про автора как пример ЛР2 лучше не открывать: в нём уже четыре формулировки и два класса.

Если спросят про правила ответа

ВОПРОС	ПРАВИЛО ОТВЕТА	ЧЬЁ	ЧТО БЕРЁТСЯ ИЗ БАЗЫ
Что такое ...?	`lr_message_about_entity`, приоритет 2	штатное	примечание `nrel_note`
На что декомпозируется ...?	`lr_message_about_subdividing`, приоритет 1	штатное	разбиение `nrel_subdividing`
Кто создал ...?	`lr_answer_about_author`, приоритет 2	**своё**	автор `nrel_language_author`

> **СКАЗАТЬ ВСЛУХ**
> Для двух штатных классов правила ответа и фразы уже были в NIKA. Для своего класса их не было, поэтому правило ответа и две фразы я написал заранее — формально это работа третьей лабораторной.

Про Wit.ai — сказать самому

> **СКАЗАТЬ ВСЛУХ**
> Wit.ai не использовался: штатный токен сервис отвергает с ошибкой no-auth. Классификация сделана логическими правилами — это штатный механизм NIKA, и в программе обработки сообщения он идёт первым, раньше Wit.ai.

И про ограничение: текст сравнивается посимвольно, с маленькой буквы или без знака вопроса вопрос не распознаётся.

20. Что показывать на ЛР3

Тема — правила ответа и фразы. Начинать с пары «вопрос один — ответ разный».

Диалог

```
Какая реализация у Python?
Какая реализация у Haskell?
Какой стандарт у Си?
Какой стандарт у Python?
Кто сопровождает Java?
Кто сопровождает JavaScript?
В каком году создан Си++?
Кто создал Haskell?
```

После первых двух вопросов сказать: «Формулировка одна, ответы разные — потому что у Python реализация в базе записана, а у Haskell нет».

Что открыть в sc-web

ИДЕНТИФИКАТОР	ЧТО СКАЗАТЬ
`python_language`, потом `haskell_language`	«У Python есть „реализация языка*“, у Haskell её нет».
`lr_answer_about_implementation`	«Правило ответа. Четыре условия: это сообщение, это вопрос о реализации, есть сущность, у сущности есть реализация. Правило не пишет текст, а указывает класс фраз».
`concept_phrase_about_implementation`	«Две фразы с пропусками. По структуре-образцу из базы берутся название языка и название реализации».
`lr_answer_about_unknown_language_property`	«Правило третьего приоритета. Когда данных нет, правило второго не срабатывает — отвечает оно».
`concept_message_about_language_property`	«Общий класс для пяти типов вопроса. По нему и срабатывает правило „сведений нет“».
`lr_classify_implementation_of_python_lang_message`	«Правило распознавания: четыре формулировки, в следствии частный класс, общий класс и сущность».

Главная мысль ЛР3

> **СКАЗАТЬ ВСЛУХ**
>
> Ответ строится в два шага. Логическое правило проверяет класс сообщения, сущность и наличие данных и указывает класс фраз. Затем по структуре-образцу фразы из базы берутся значения. Проверка наличия данных стоит в самом правиле, потому что вывод останавливается на первом сработавшем правиле: без неё на вопрос о языке без нужных сведений ответа не было бы вовсе.

21. Вопросы на стыке двух лаб

То, что могут спросить именно потому, что обе лабы в одной системе.

Почему правил 100, а в отчёте ЛР2 — 67?

67 было на момент второй лабораторной. В третьей семь правил про автора заменены расширенными и добавлены 33 новых.

Зачем менялись правила про автора?

Чтобы вопрос об авторе вёл себя как остальные четыре типа: общий класс на сообщении нужен правилу «сведений нет». Заодно добавлены две формулировки и восьмой язык.

Где граница между лабами?

ЛР2: правила `lr_classify_what_is_*` и `lr_classify_subdividing_of_*`, класс сообщения об авторе, правило ответа и фразы об авторе. ЛР3: правила `lr_classify_<тип>_of_<язык>`, четыре класса сообщений, общий класс, четыре правила ответа, правило «сведений нет» и их фразы.

Почему для каждого понятия отдельное правило?

Правило сопоставляет конкретный текст с конкретным узлом базы и обобщать не умеет. Обобщение — задача Wit.ai, а он недоступен. В штатной базе NIKA сделано так же.

Почему в ответе «компилятор - Компилятор — ...» и две точки?

Это штатная фраза NIKA: название, дефис, примечание и точка. Примечание в базе само начинается с названия и заканчивается точкой.

Что за Error в журнале на вопрос «Что такое ...?»

Это система взяла первой штатную фразу с определением. У наших понятий отношения `nrel_definition` нет, фраза не собирается, и берётся вторая — с примечанием. Ответ при этом есть.

Чем «сведений нет» отличается от «Я не понимаю Вас»?

Второе — штатное, выдаётся, когда вопрос не распознан вовсе. Первое — из третьей лабораторной: вопрос распознан, тип и язык известны, но нужного отношения у языка в базе нет.

Почему общий класс ставится правилом, а не выводится из включения?

Принадлежность классу по отношению включения в sc-памяти сама не распространяется. Правило «сведений нет» ищет дугу от общего класса к сообщению, поэтому её приходится создавать явно.

Подробные ответы на вопросы преподавателя и разбор задач — в шпаргалках: `ЛР2/Шпаргалка_к_защите_ЛР2.pdf` и `ЛР3/Шпаргалка_к_защите_ЛР3.pdf`.